University of Chittagong

Department of Computer Science and Engineering

Healthcare Web Application

D2: Design Document

CSE-616: Web Engineering Lab

Submitted to,

Rokan Uddin Faruqui

Professor
Department of Computer Science and Engineering
University of Chittagong
Chittagong-4331, Bangladesh

Submitted by,

Group - 11		
Sl.	**Name**	**Student ID**
1	Shahrukh Ahmed Meherab	22701006
2	Md. Emranul Hoque	22701038
3	Abu Ruhan Mahamud	22701074

Date of Submission: October 22, 2025

Contents

1 System Architecture **3**
- 1.1 Overview . . . 3
- 1.2 High-Level Architecture . . . 3
- 1.3 Architecture Components . . . 3
  - 1.3.1 Presentation Tier (Frontend) . . . 3
  - 1.3.2 Application Tier (Backend) . . . 4
  - 1.3.3 Data Tier (Database) . . . 4
- 1.4 Request Flow Example . . . 4

2 Database Design **6**
- 2.1 Overview . . . 6
- 2.2 Entity-Relationship Diagram . . . 6
- 2.3 Entity Relationships . . . 6
- 2.4 Relational Schema . . . 8
  - 2.4.1 User Authentication Schema . . . 8
  - 2.4.2 Patient and Doctor Schema . . . 8
  - 2.4.3 Appointment and Medical Record Schema . . . 9
  - 2.4.4 Prescription and Medication Schema . . . 10
  - 2.4.5 Medical Document Schema . . . 11

3 API Specification **12**
- 3.1 Overview . . . 12
- 3.2 Authentication Endpoints . . . 12
- 3.3 Patient Endpoints . . . 12
- 3.4 Doctor Endpoints . . . 12
- 3.5 Appointment Endpoints . . . 12
- 3.6 Medical Record Endpoints . . . 12
- 3.7 Prescription Endpoints . . . 13
- 3.8 Medication Endpoints . . . 13
- 3.9 Medical Document Endpoints . . . 13
- 3.10 Admin Endpoints . . . 13
- 3.11 Response Codes . . . 13

4 UI/UX Mockups **14**

5 Security Architecture **18**
- 5.1 Overview . . . 18
- 5.2 Authentication . . . 18
- 5.3 Authorization . . . 18
  - 5.3.1 Role-Based Access Control . . . 18
  - 5.3.2 Access Control Matrix . . . 18
- 5.4 Data Protection . . . 19
  - 5.4.1 Encryption . . . 19

5.4.2 Input Validation . . . 19
5.5 Protection Against Common Vulnerabilities . . . 19
5.5.1 SQL Injection Prevention . . . 19
5.5.2 Unauthorized Access Prevention . . . 20
5.6 Additional Security Measures . . . 20
5.7 Summary . . . 20

List of Figures

1 Three-Tier System Architecture . . . 3
2 Request Flow for Appointment Booking . . . 5
3 Entity-Relationship Diagram for Healthcare Management System . . . 7
4 Registration Page Design . . . 14
5 Login Page Design . . . 14
6 Homepage Design . . . 15
7 Appointment Page Design . . . 16
8 Patient Page Design . . . 16
9 Admin Dashboard Design . . . 17

List of Tables

1 Entity Relationships and Cardinalities . . . 6
2 User Role Permissions . . . 19

1 System Architecture

1.1 Overview

The healthcare web application follows a three-tier architecture pattern that separates the presentation layer, application logic layer, and data storage layer. This design provides clear separation of concerns, enhances maintainability, and supports independent development of frontend and backend components. The architecture enables parallel development, simplifies testing, and allows each tier to be updated independently.

1.2 High-Level Architecture

Figure 1 illustrates the three-tier architecture of our system.

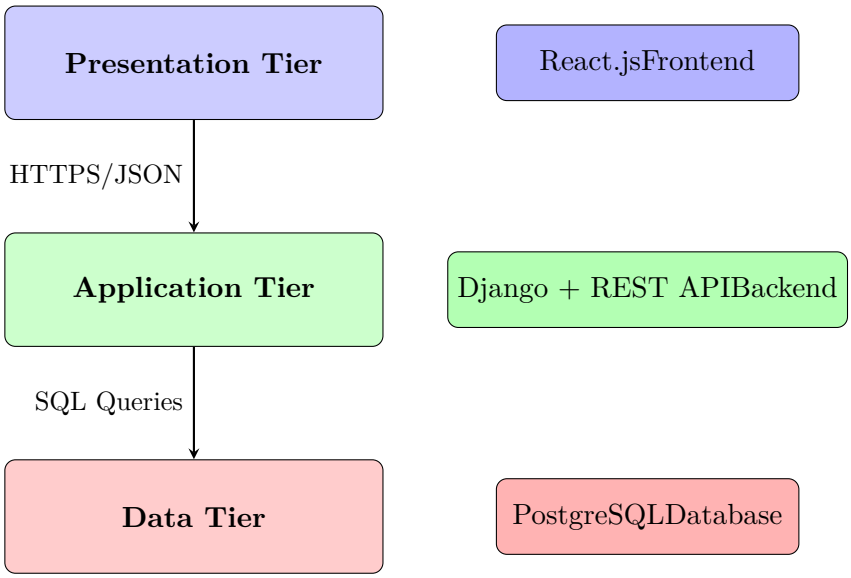

Figure 1: Three-Tier System Architecture

1.3 Architecture Components

1.3.1 Presentation Tier (Frontend)

The presentation tier is built using React.js and runs in the user's web browser. It provides three separate interfaces for different user types: patients, doctors, and administrators.

Key Responsibilities:

- Display user interface components
- Handle user interactions and form inputs
- Validate user input before sending to backend
- Make asynchronous API calls to backend
- Update interface dynamically based on server responses

Technology: React.js for component-based UI development and responsive single-page applications.

1.3.2 Application Tier (Backend)

The application tier is implemented using Django and Django REST Framework. It handles all business logic, authentication, and data processing.

Key Components:

- **URL Router:** Routes incoming requests to appropriate handlers
- **Authentication System:** Validates user tokens and manages sessions
- **Views:** Implement business logic and handle request processing
- **Serializers:** Convert data between JSON and Python objects
- **Models:** Define database structure using Django ORM
- **Permissions:** Enforce role-based access control

Technology: Django and Django REST Framework for secure backend with built-in authentication, ORM, and robust API capabilities.

1.3.3 Data Tier (Database)

The data tier uses PostgreSQL to store all application data persistently.

Key Responsibilities:

- Store user accounts, patient profiles, medical records
- Execute SQL queries and return results
- Maintain data integrity through constraints
- Handle transactions and concurrent access

Technology: PostgreSQL for reliable data storage with advanced features like JSON fields and array types useful for medical data.

1.4 Request Flow Example

Figure 2 illustrates the complete request flow when a patient books an appointment.

This flow demonstrates how the three tiers work together: React handles user interaction, Django enforces business rules and validates permissions, and PostgreSQL ensures data persistence.

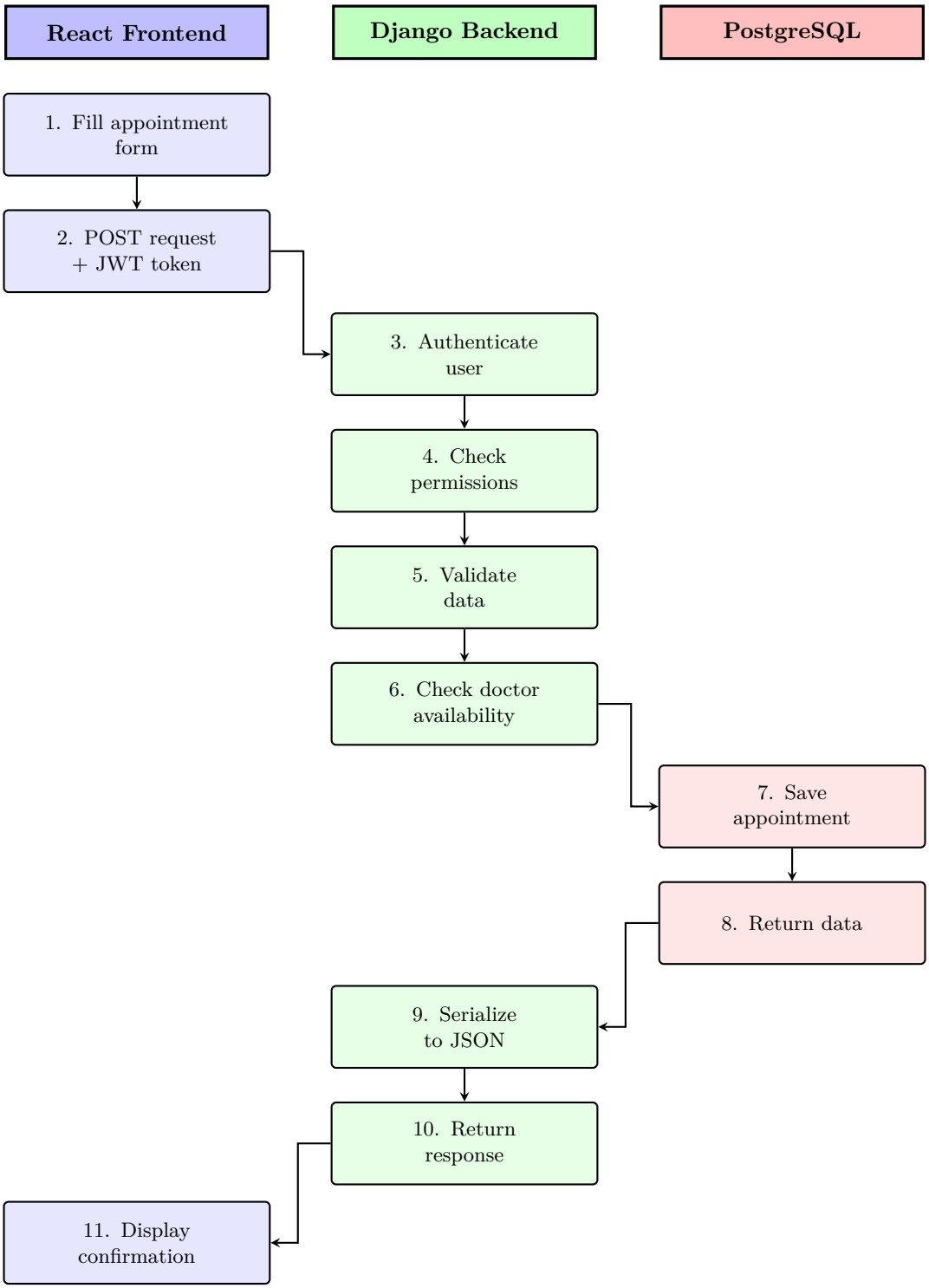

Figure 2: Request Flow for Appointment Booking

2 Database Design

2.1 Overview

The database architecture for the healthcare web application follows a normalized relational model designed to support comprehensive medical data management while ensuring data integrity, security, and scalability.

The design adheres to third normal form (3NF) to eliminate data redundancy and prevent update anomalies, which is critical in healthcare applications where data accuracy directly impacts patient care. All tables include timestamp fields (created_at, updated_at) for audit trails, and foreign key constraints with appropriate CASCADE and SET NULL behaviors ensure referential integrity throughout the system.

Our database design supports three primary user workflows: (1) patient self-service operations including appointment booking and medical history access, (2) doctor clinical workflows encompassing patient record management and prescription writing, and (3) administrative functions for user management and system monitoring. The schema maintains clear separation between authentication data (users table) and role-specific profile information (patients and doctors tables) to support future extensibility.

2.2 Entity-Relationship Diagram

Figure 3 illustrates the database structure for the healthcare system. The design centers around a User entity that serves as the authentication base, with Patient and Doctor entities inheriting from it through an ISA relationship to store role-specific profile data. The schema supports core workflows: patients booking appointments with doctors, doctors creating medical records and prescriptions after appointments, and patients tracking their current medications with reminders. Additional entities handle medical document uploads. This nine-entity design implements all functional requirements from the requirements document while maintaining third normal form to ensure data integrity and minimize redundancy.

2.3 Entity Relationships

Table 1 summarizes the relationships between entities and their cardinalities.

From Entity	**Relationship**	**To Entity**	**Cardinality**
User	ISA	Patient	1:1 (inheritance)
User	ISA	Doctor	1:1 (inheritance)
Patient	books	Appointment	1:N
Doctor	schedules	Appointment	1:N
Appointment	has	MedicalRecord	1:1
MedicalRecord	contains	Prescription	1:N
Patient	takes	Medication	1:N
Medication	triggers	Reminder	1:N
Patient	uploads	MedicalDocument	1:N

Table 1: Entity Relationships and Cardinalities

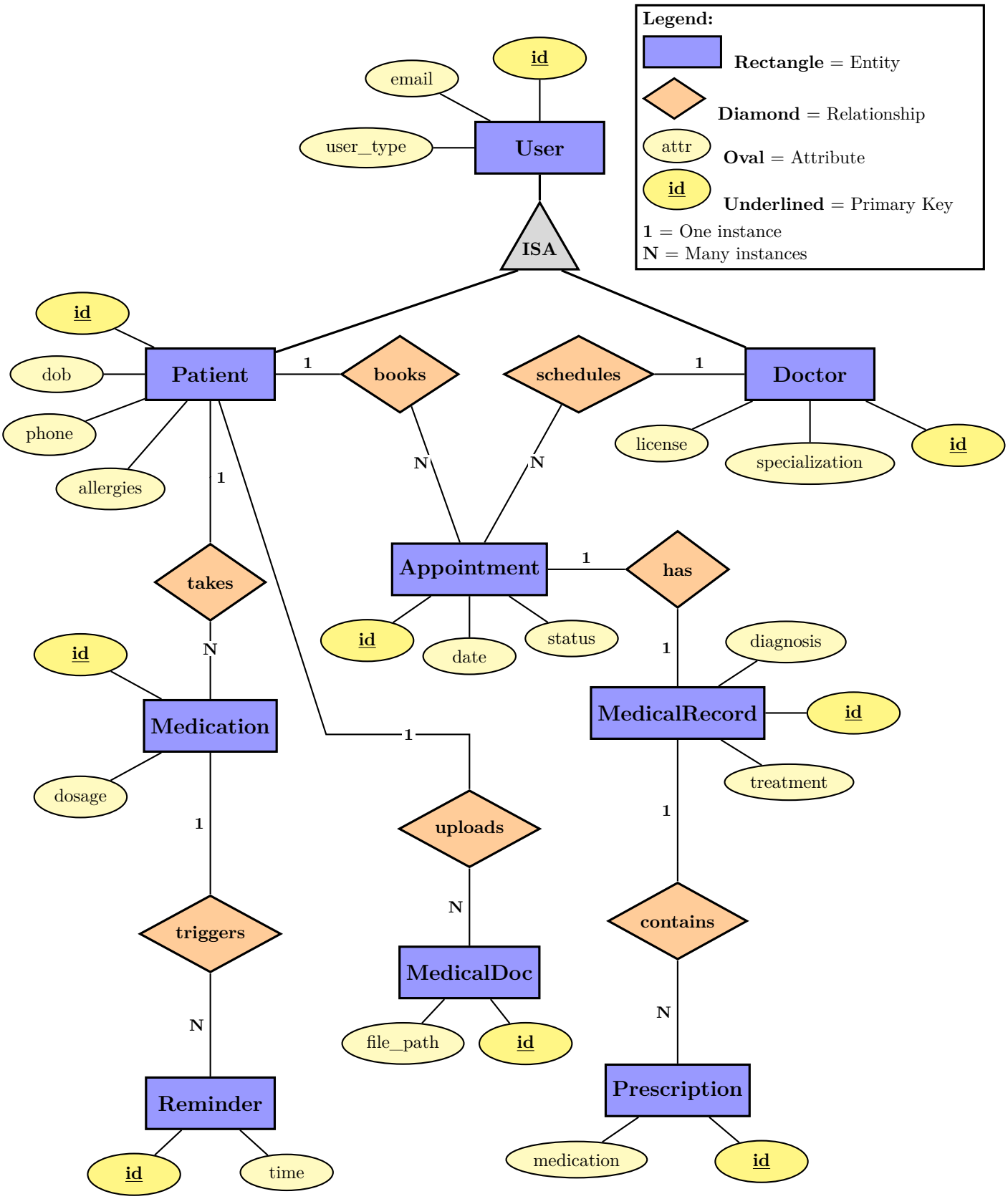

Figure 3: Entity-Relationship Diagram for Healthcare Management System

2.4 Relational Schema

2.4.1 User Authentication Schema

```sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    user_type VARCHAR(20) NOT NULL
        CHECK (user_type IN ('patient', 'doctor', 'admin')),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);
```

Listing 1: User Table Schema

2.4.2 Patient and Doctor Schema

```sql
CREATE TABLE patients (
    id SERIAL PRIMARY KEY,
    user_id INTEGER UNIQUE NOT NULL
        REFERENCES users(id) ON DELETE CASCADE,
    date_of_birth DATE NOT NULL,
    phone_number VARCHAR(20),
    emergency_contact_name VARCHAR(100),
    emergency_contact_phone VARCHAR(20),
    allergies TEXT[] DEFAULT '{}',
    medical_history JSONB DEFAULT '{}',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_patients_user ON patients(user_id);
```

Listing 2: Patient Table Schema

```sql
CREATE TABLE doctors (
    id SERIAL PRIMARY KEY,
    user_id INTEGER UNIQUE NOT NULL
        REFERENCES users(id) ON DELETE CASCADE,
    license_number VARCHAR(50) UNIQUE NOT NULL,
```

```sql
    specialization VARCHAR(100) NOT NULL,
    years_experience INTEGER DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_doctors_license ON doctors(license_number);
```

Listing 3: Doctor Table Schema

2.4.3 Appointment and Medical Record Schema

```sql
CREATE TABLE appointments (
    id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL
        REFERENCES patients(id) ON DELETE CASCADE,
    doctor_id INTEGER NOT NULL
        REFERENCES doctors(id) ON DELETE CASCADE,
    appointment_date TIMESTAMP NOT NULL,
    status VARCHAR(20) DEFAULT 'scheduled'
        CHECK (status IN ('scheduled', 'confirmed',
                          'completed', 'cancelled')),
    reason TEXT,
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_doctor_time
        UNIQUE(doctor_id, appointment_date)
);

CREATE INDEX idx_appointments_patient ON appointments(patient_id);
CREATE INDEX idx_appointments_doctor ON appointments(doctor_id);
CREATE INDEX idx_appointments_date ON appointments(appointment_date)
    ;
```

Listing 4: Appointment Table Schema

```sql
CREATE TABLE medical_records (
    id SERIAL PRIMARY KEY,
    appointment_id INTEGER UNIQUE NOT NULL
        REFERENCES appointments(id) ON DELETE CASCADE,
    doctor_id INTEGER NOT NULL REFERENCES doctors(id),
    diagnosis TEXT NOT NULL,
    symptoms TEXT,
    treatment_notes TEXT,
```

```sql
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_medical_records_appointment
    ON medical_records(appointment_id);
```

Listing 5: Medical Record Table Schema

2.4.4 Prescription and Medication Schema

```sql
CREATE TABLE prescriptions (
    id SERIAL PRIMARY KEY,
    medical_record_id INTEGER NOT NULL
        REFERENCES medical_records(id) ON DELETE CASCADE,
    medication_name VARCHAR(200) NOT NULL,
    dosage VARCHAR(100) NOT NULL,
    frequency VARCHAR(100) NOT NULL,
    duration VARCHAR(100),
    instructions TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_prescriptions_record
    ON prescriptions(medical_record_id);
```

Listing 6: Prescription Table Schema

```sql
CREATE TABLE medications (
    id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL
        REFERENCES patients(id) ON DELETE CASCADE,
    prescription_id INTEGER
        REFERENCES prescriptions(id) ON DELETE SET NULL,
    name VARCHAR(200) NOT NULL,
    dosage VARCHAR(100) NOT NULL,
    frequency VARCHAR(100) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE,
    is_active BOOLEAN DEFAULT true
);

CREATE TABLE medication_reminders (
    id SERIAL PRIMARY KEY,
    medication_id INTEGER NOT NULL
```

```sql
        REFERENCES medications(id) ON DELETE CASCADE,
    reminder_time TIME NOT NULL,
    is_enabled BOOLEAN DEFAULT true,
    last_reminded_at TIMESTAMP
);

CREATE INDEX idx_medications_patient ON medications(patient_id);
CREATE INDEX idx_reminders_medication
    ON medication_reminders(medication_id);
```

Listing 7: Medication and Reminder Tables

2.4.5 Medical Document Schema

```sql
CREATE TABLE medical_documents (
    id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL
        REFERENCES patients(id) ON DELETE CASCADE,
    uploaded_by_doctor_id INTEGER REFERENCES doctors(id),
    document_type VARCHAR(50) NOT NULL
        CHECK (document_type IN ('X-ray', 'MRI',
                                 'Lab Report', 'Other')),
    file_path VARCHAR(500) NOT NULL,
    description TEXT,
    upload_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_medical_docs_patient
    ON medical_documents(patient_id);
```

Listing 8: Medical Document Table Schema

3 API Specification

3.1 Overview

The healthcare application provides RESTful API endpoints for communication between the React frontend and Django backend. All endpoints use JSON for data exchange and require token-based authentication except for registration and login.

3.2 Authentication Endpoints

1. **POST /api/auth/register/** : Register a new user account. No authentication required.
2. **POST /api/auth/login/** : Authenticate user and receive access token. No authentication required.
3. **POST /api/auth/logout/** : Invalidate current authentication token. Authentication required.

3.3 Patient Endpoints

1. **GET /api/patients/<id>/** : Retrieve patient profile. Patient can view own profile, doctors can view patient profiles.
2. **PATCH /api/patients/<id>/** : Update patient profile information. Only the patient can update their own profile.

3.4 Doctor Endpoints

1. **GET /api/doctors/** : Get list of all doctors for appointment booking. Optional query parameter: `specialization`.
2. **GET /api/doctors/<id>/** : Retrieve specific doctor profile information.

3.5 Appointment Endpoints

1. **GET /api/appointments/** : Get appointments for logged-in user. Patients see their appointments, doctors see appointments with them. Optional query parameters: `status`, `date`.
2. **POST /api/appointments/** : Book a new appointment. Only patients can create appointments.
3. **PATCH /api/appointments/<id>/** : Update appointment status or details. Patients can cancel/reschedule, doctors can update status.

3.6 Medical Record Endpoints

1. **GET /api/patients/<patient_id>/medical-records/** : Get all medical records for a patient. Patient can view own records, doctors can view patient records.
2. **POST /api/medical-records/** : Create medical record after appointment. Only doctors can create records.

3.7 Prescription Endpoints

1. **POST /api/prescriptions/** : Create prescription as part of medical record. Only doctors can create prescriptions.
2. **GET /api/patients/<patient_id>/prescriptions/** : Get all prescriptions for a patient.

3.8 Medication Endpoints

1. **GET /api/medications/** : Get current medications for logged-in patient. Optional query parameter: `is_active` (default: true).
2. **POST /api/medications/** : Add medication to patient's tracking list. Only patients can add.

3.9 Medical Document Endpoints

1. **POST /api/medical-documents/** : Upload medical document (X-ray, PDF, etc.). Patients and doctors can upload. Request uses multipart/form-data with fields: `patient`, `document_type`, `file`, `description`.
2. **GET /api/patients/<patient_id>/documents/** : Get all medical documents for a patient with download URLs.

3.10 Admin Endpoints

1. **GET /api/admin/users/** : Get list of all system users. Admin only.
2. **PATCH /api/admin/users/<id>/deactivate/** : Deactivate a user account. Admin only.

3.11 Response Codes

- `200 OK`: Successful request
- `201 Created`: Resource created
- `400 Bad Request`: Invalid data
- `401 Unauthorized`: Authentication required
- `403 Forbidden`: Insufficient permissions
- `404 Not Found`: Resource not found
- `409 Conflict`: Request conflicts with current state

4 UI/UX Mockups

The user interface design focuses on creating intuitive and accessible interfaces for users. All mockups were created using Figma and emphasize clean layouts, clear navigation, and responsive design principles.

Figma Project: Healthcare Web Application UI

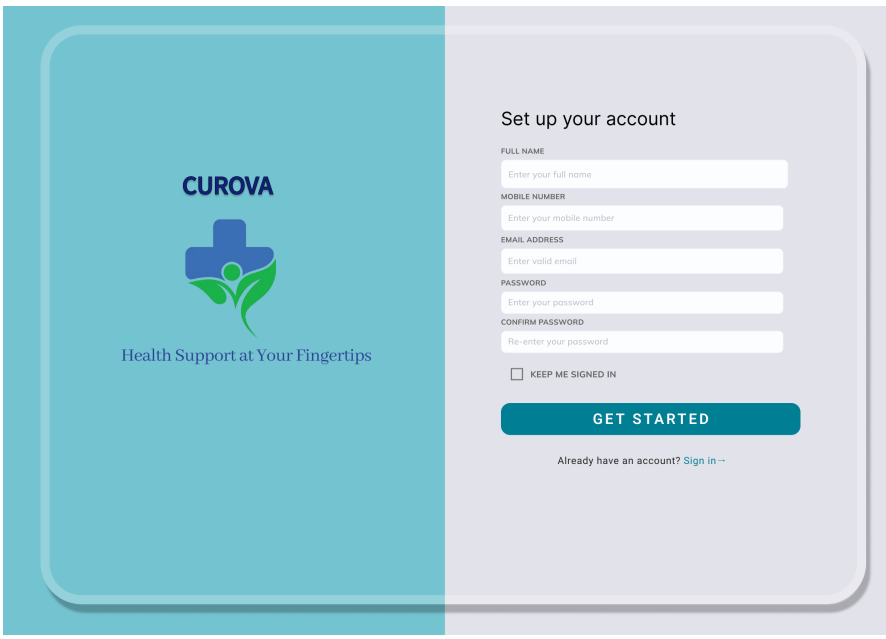

Figure 4: Registration Page Design

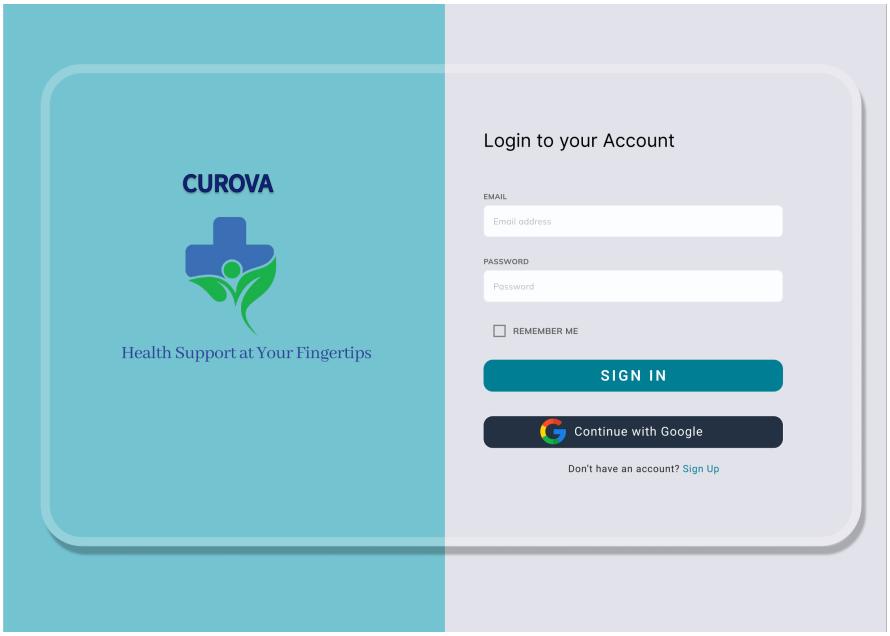

Figure 5: Login Page Design

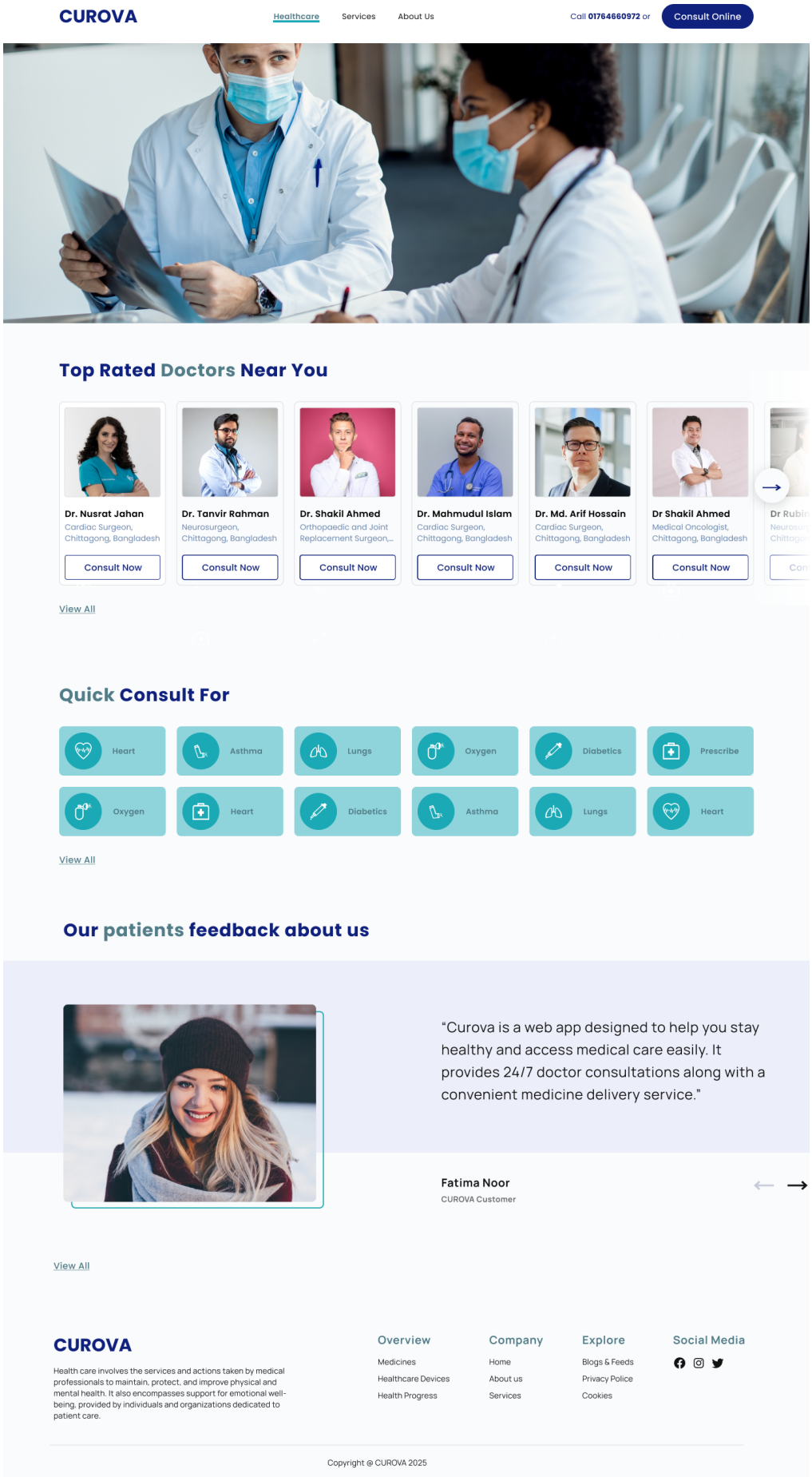

Figure 6: Homepage Design

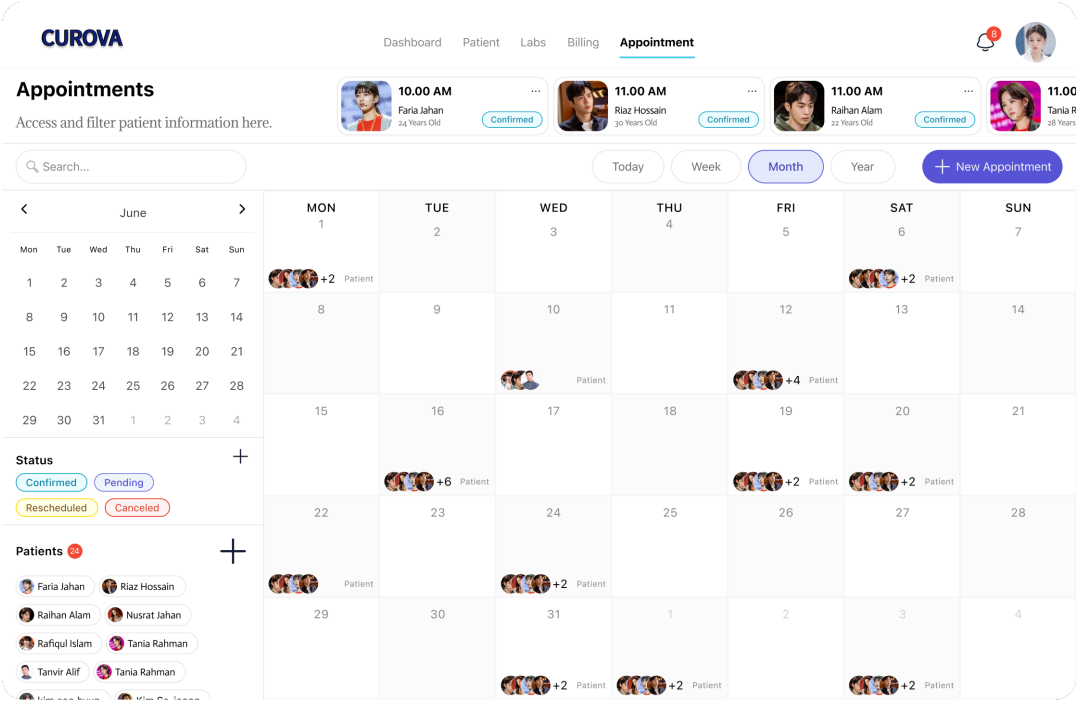

Figure 7: Appointment Page Design

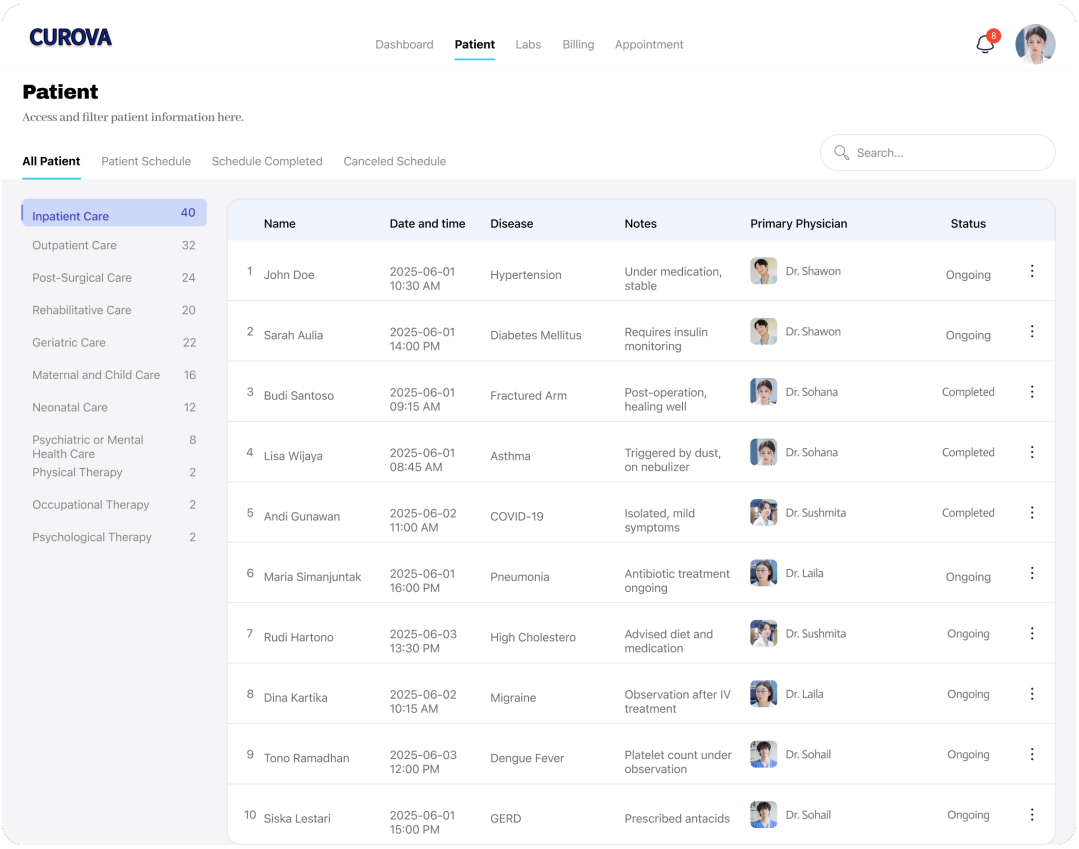

Figure 8: Patient Page Design

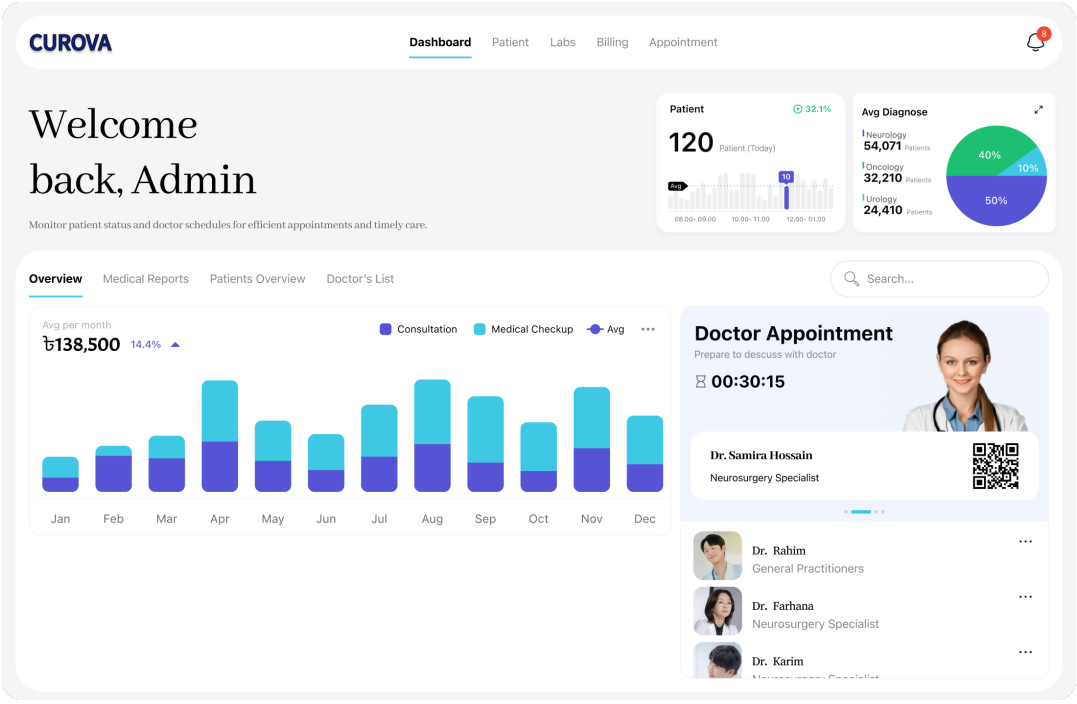

Figure 9: Admin Dashboard Design

5 Security Architecture

5.1 Overview

The healthcare application implements security measures to protect patient data through authentication, authorization, data encryption, and protection against common vulnerabilities. This design addresses the security requirements specified in our requirements document.

5.2 Authentication

The system uses token-based authentication provided by Django REST Framework.
How it works:

1. User logs in with email and password
2. Backend verifies credentials and generates a token
3. Token is returned to frontend and stored in browser
4. Frontend includes token in all subsequent requests
5. Backend validates token before processing each request

Security measures:

- Passwords hashed using Django's PBKDF2 algorithm (not stored in plain text)
- All communication over HTTPS to prevent interception
- Tokens can be invalidated on logout

5.3 Authorization

Authorization controls what authenticated users can access based on their role (patient, doctor, admin).

5.3.1 Role-Based Access Control

The system implements three permission classes:

- **IsPatient**: Only patients can access patient-specific endpoints
- **IsDoctor**: Only doctors can access doctor-specific endpoints
- **IsAdmin**: Only admins can access administrative functions
- **IsOwnerOrDoctor**: Patients access own data, doctors access patient data

5.3.2 Access Control Matrix

Table 2 shows what each user role can do.
Example: When a patient tries to access `/api/patients/10/`, the system checks if they own that patient record. If not, it returns `403 Forbidden`.

Resource	Patient	Doctor	Admin
Own Profile	Read, Update	-	-
Other Profiles	-	Read	Read
Appointments	Create, Read, Cancel	Read, Update	-
Medical Records	Read (own)	Create, Read	-
Prescriptions	Read (own)	Create	-
Documents	Upload, Read (own)	Upload, Read	-
User Management	-	-	Full Access

Table 2: User Role Permissions

5.4 Data Protection

5.4.1 Encryption

Data in Transit:

- HTTPS required for all communication between browser and server
- Prevents eavesdropping and tampering with data

Data at Rest:

- Passwords stored hashed, never in plain text
- Database credentials stored in environment variables (not in code)
- Sensitive files excluded from version control using .gitignore

5.4.2 Input Validation

User inputs are validated to ensure data quality and prevent malicious entries:

- Django serializers validate data types and required fields
- Email format checked during registration
- Date/time format validated for appointments
- File uploads restricted to PDF, JPG, PNG only
- Maximum file size limit of 5MB

5.5 Protection Against Common Vulnerabilities

As required by the capstone guidelines, the system protects against at least two common vulnerabilities.

5.5.1 SQL Injection Prevention

What it is: Attackers try to inject malicious database commands through user inputs.
How we prevent it: Django ORM automatically uses parameterized queries. User inputs are never directly inserted into SQL statements.
Example:

```
# Safe: Django ORM parameterized query
Patient.objects.filter(phone_number=user_input)

# Unsafe: Never used in our system
"SELECT * FROM patients WHERE phone='" + user_input + "'"
```

5.5.2 Unauthorized Access Prevention

What it is: Users trying to access data they shouldn't by changing URLs or request parameters.
How we prevent it:

- All endpoints except login/register require authentication
- Permission classes check user role before processing requests
- Object-level checks verify ownership (patients can't see other patients' data)
- Invalid access attempts return `403 Forbidden`

Example: Patient changes URL from `/api/patients/5/` to `/api/patients/10/`. Backend verifies ownership and blocks access if the patient doesn't own record 10.

5.6 Additional Security Measures

Cross-Site Request Forgery (CSRF) Protection:

- Django CSRF middleware enabled
- CSRF tokens required for state-changing operations
- CORS configured to allow requests only from trusted frontend

Cross-Site Scripting (XSS) Protection:

- Django automatically escapes output
- React escapes variables by default
- User-generated content sanitized

5.7 Summary

The security architecture protects patient data through multiple layers: token-based authentication verifies user identity, role-based authorization controls access to resources, HTTPS encryption protects data during transmission, and input validation prevents malicious data entry. The system implements protections against SQL injection and unauthorized access as required by the capstone guidelines.